Chapter 6: Performance Considerations 3

CSCD439/539 GPU Programming

Additional “memories”

`__constant__`

- Read-only
- Data resides in global memory
- Different read path:
  - includes specialized caches

Constant Memory

- Data stored in global memory, read through a constant-cache path
  - `__constant__` qualifier in declarations
  - Can only be read by GPU kernels
  - Limited to 64KB
- To be used when all threads in a warp read the same address
  - Serializes otherwise
- Throughput:
  - 32 bits per warp per clock per multiprocessor

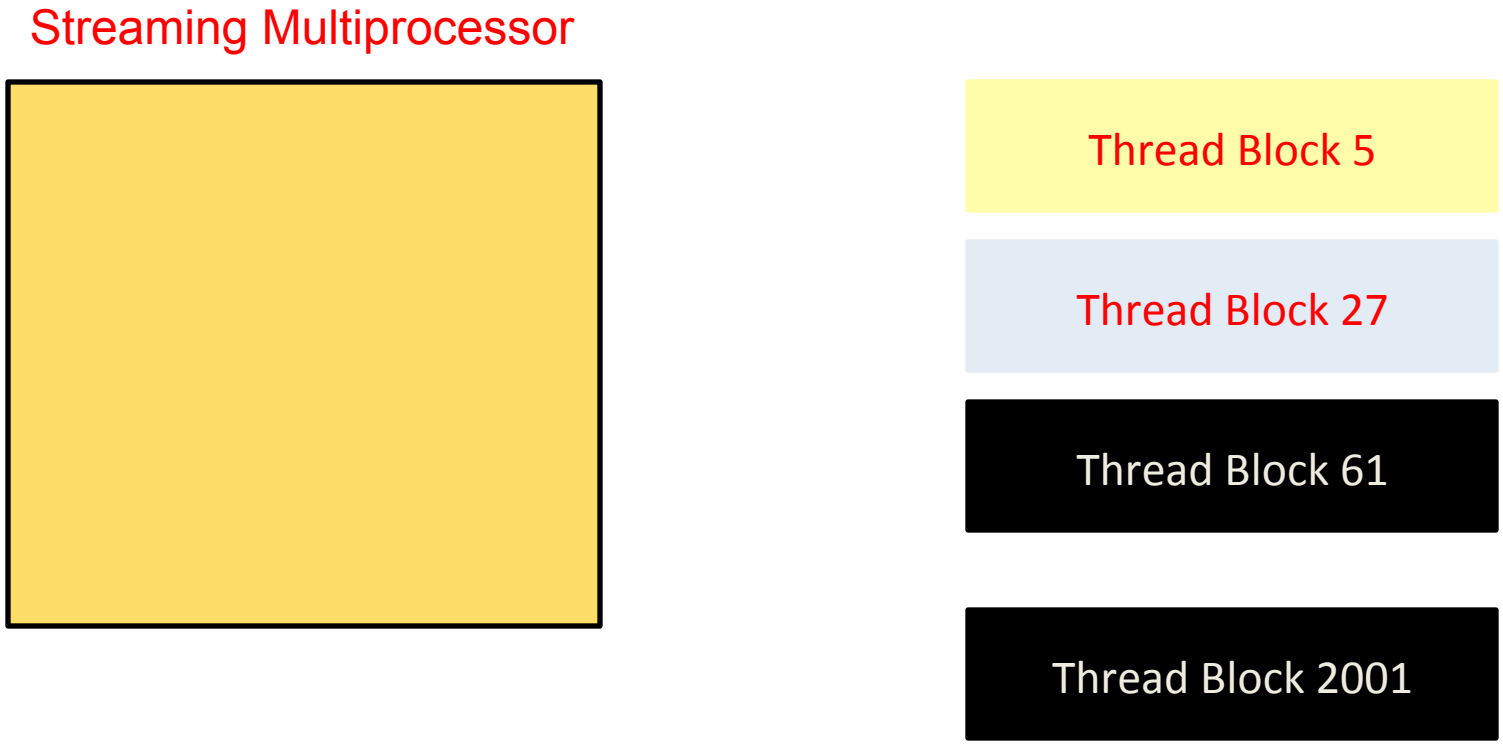
Performance Considerations

- How SM Execute Threads
- Control-Flow Divergence
- Occupancy
- Kernel Launch Overheads

How an SM executes threads

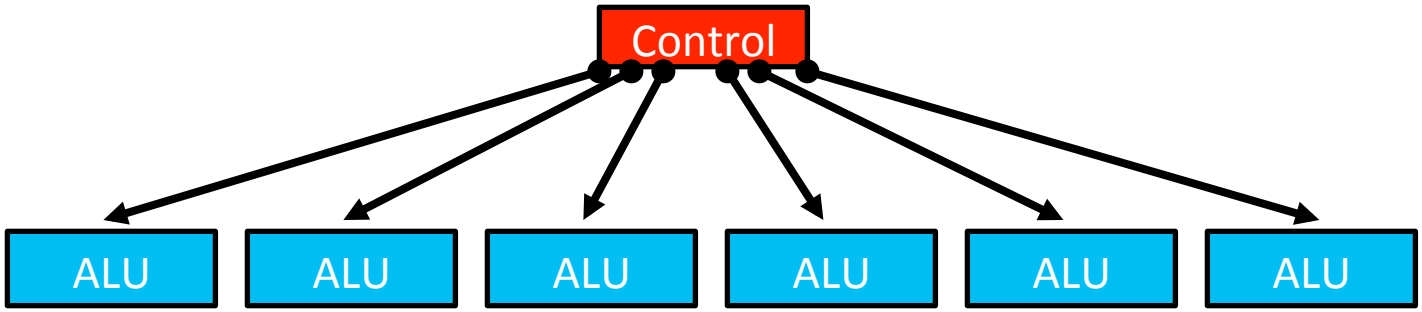
- Overview of how a Stream Multiprocessor works
- SIMT Execution
- Divergence

Scheduling Blocks onto SMs



- **HW Schedules thread blocks onto available SMs**
  - **No guarantee of ordering among thread blocks**
  - **HW will schedule thread blocks as soon as a previous thread block finishes**

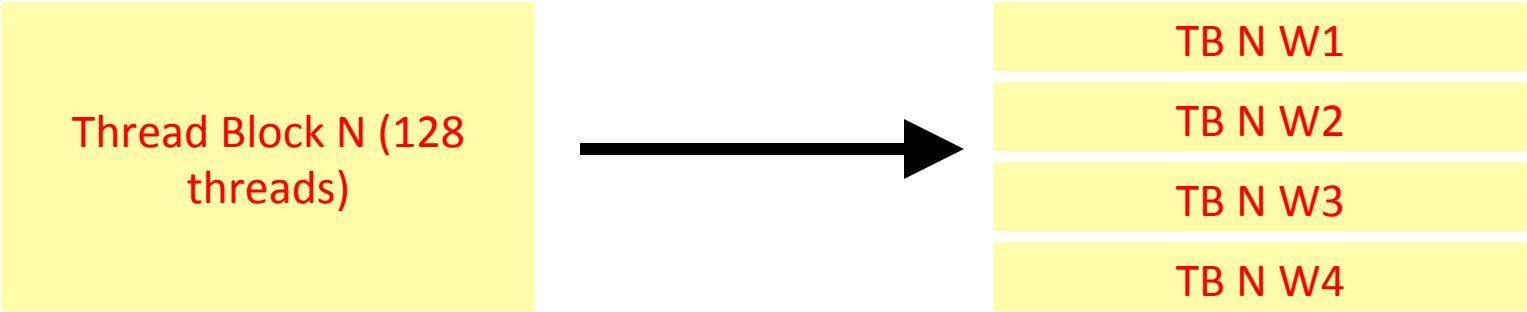
Warps



- A warp = 32 threads launched together
  - Usually, execute together as well

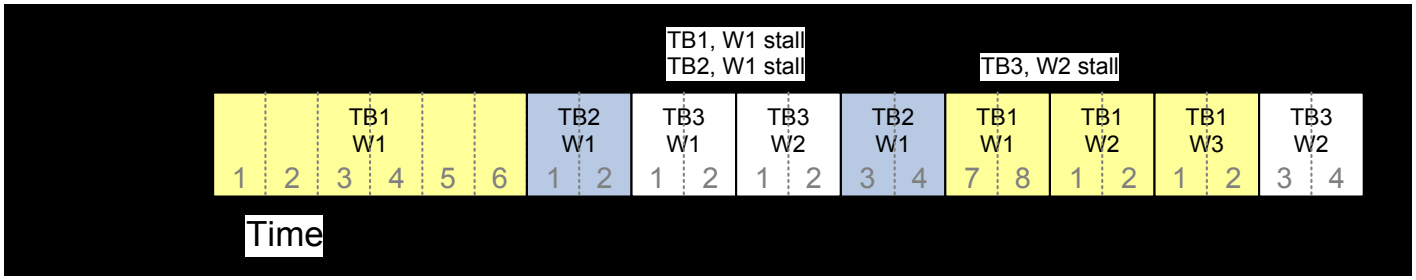
Mapping of Thread Blocks

- Each thread block is mapped to one or more warps
- The hardware schedules each warp independently



Thread Scheduling Example

- SM implements zero-overhead warp scheduling
  - For Old GPUs, at any time, only one of the warps is executed by SM * (not true any more now)
  - Warps whose next instruction has its inputs ready for consumption are eligible for execution
  - Eligible Warps are selected for execution on a prioritized scheduling policy
  - All threads in a warp execute the same instruction when selected



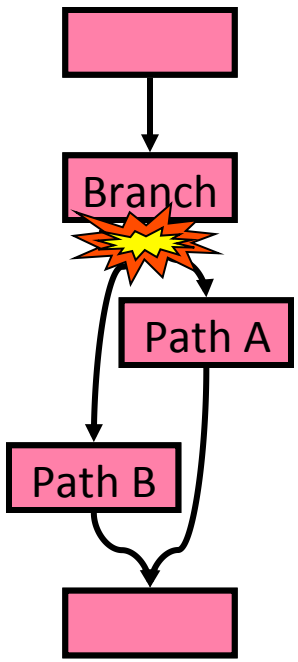
CONTROL FLOW DIVERGENCE

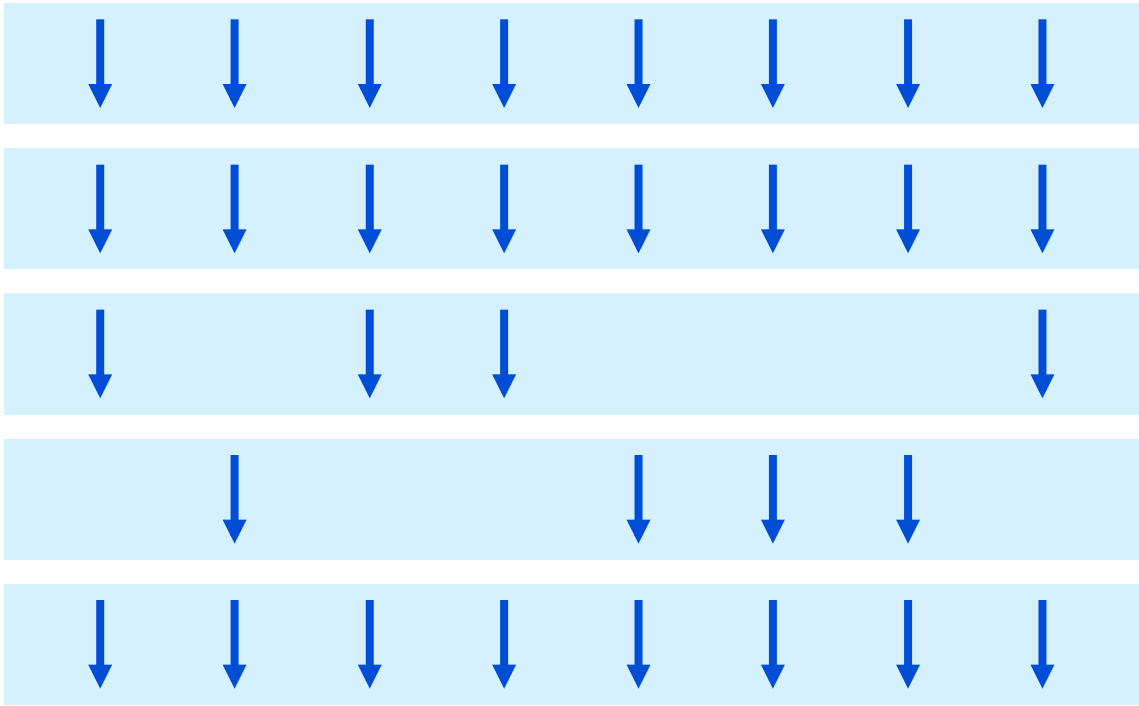
Control Flow Divergence

- What happens if you have the following code?

```
if(foo(threadIdx.x))
{
   do_A();
}
else
{
   do_B();
}
```

Control Flow Divergence





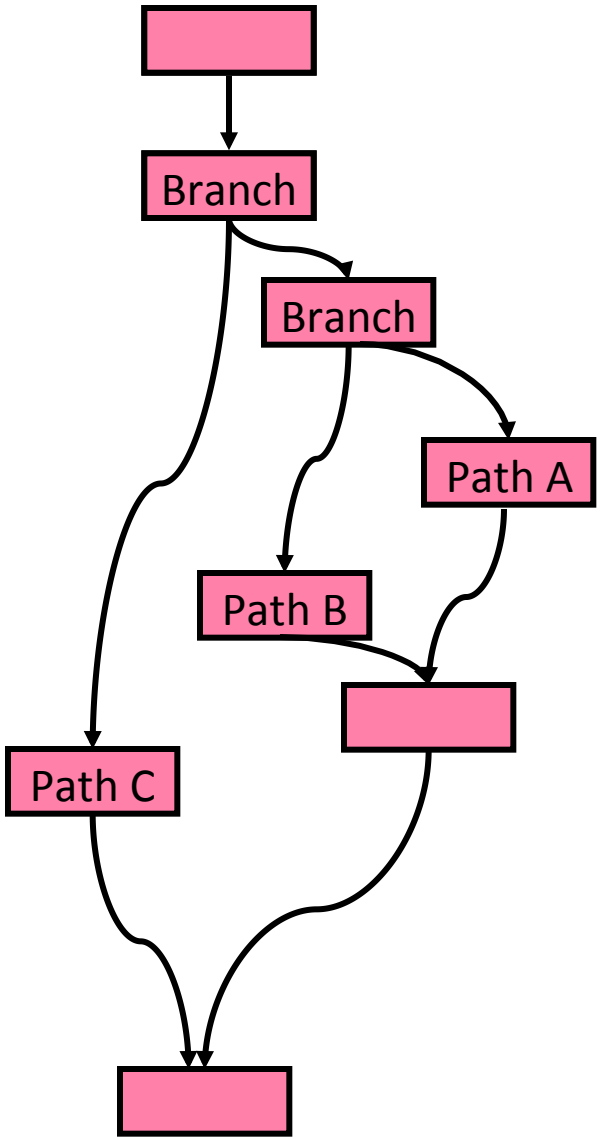
From Fung et al. MICRO '07

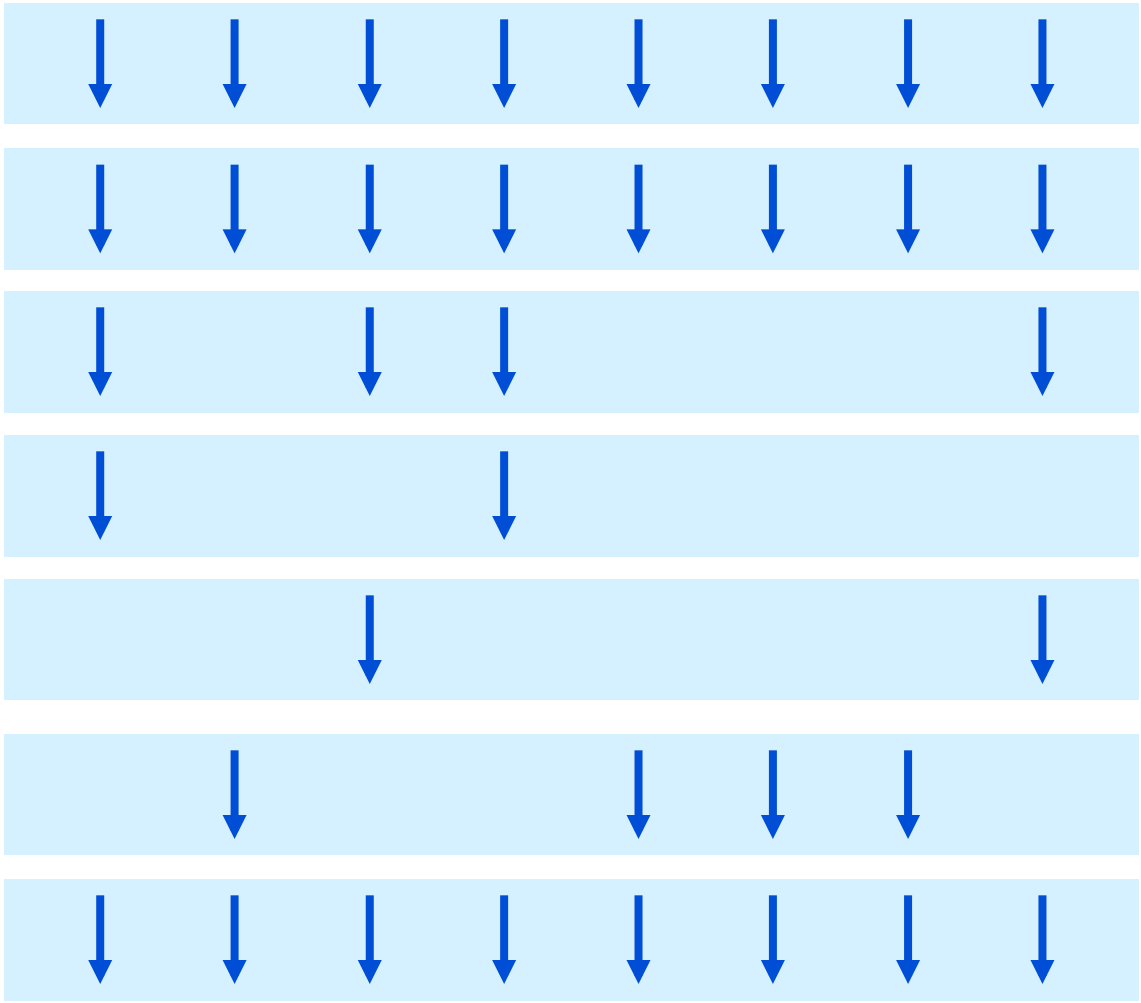
Control Flow Divergence

- Nested branches are handled as well

```
if(foo(threadIdx.x))
{
  if(bar(threadIdx.x))
    do_A();
  else
    do_B();
}
else
  do_C();
```

Control Flow Divergence





Control Flow Divergence

- You don’t have to worry about divergence for correctness (*)
- You might have to think about it for performance
  - Depends on your branch conditions

Control Flow Divergence

- Instructions are issued per 32 threads (warp)
- Divergent branches:
  - Threads within a single warp take different paths
    - `if-else`, ...
  - Different execution paths within a warp are serialized
- Different warps can execute different code with no impact on performance

Control Flow

- Avoid diverging within a warp
  - Example with divergence:

    ```
    if (threadIdx.x > 2) {...}
    else {...}
    ```

    Branch granularity < warp size
  - Example without divergence:

    ```
    if (threadIdx.x / WARP_SIZE > 2)
    {...}
    else {...}
    ```

    Branch granularity is a whole multiple of warp size

Control Flow Example

- Avoid diverging within a warp
  - Example with divergence:

```
__shared__ float partialSum[]
...
unsigned int t = threadIdx.x;
for (unsigned int stride = 1;  stride < blockDim.x; stride *= 2)
{
    __syncthreads();
    if (t % (2*stride) == 0)
        partialSum[t] += partialSum[t+stride];
}
```

FIGURE 6.2

A simple sum reduction kernel.

Control Flow Example

- Avoid diverging within a warp
  - Example with divergence:

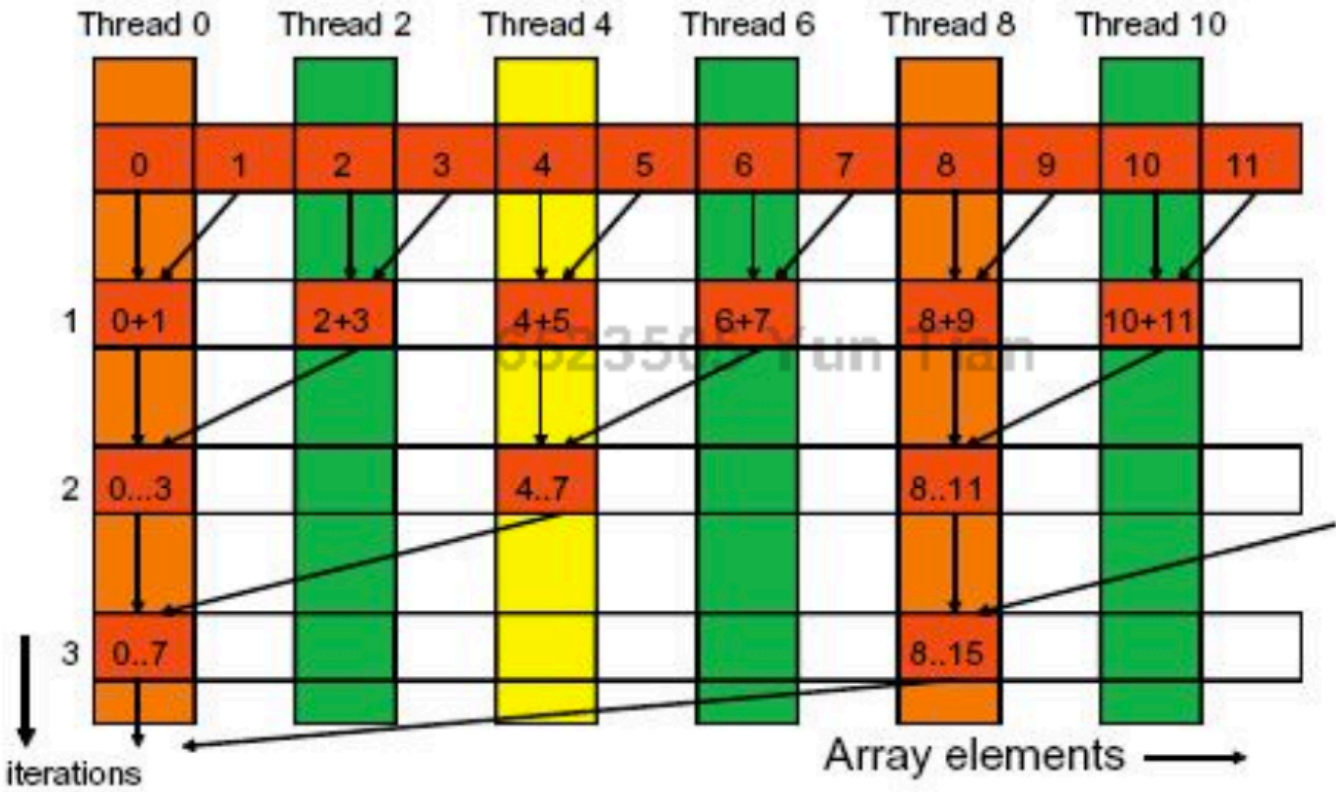


FIGURE 6.3

Execution of the sum reduction kernel.

Control Flow Example

- Avoid diverging within a warp
  - Example with fewer divergence:

```
__shared__ float partialSum[]

unsigned int t = threadIdx.x;
for (unsigned int stride = blockDim.x; stride >= 1; stride /= 2)
{
    __syncthreads();
    if (t < stride)
     partialSum[t] += partialSum[t+stride];
}
```

FIGURE 6.4

A kernel with fewer thread divergence.

Control Flow Example

- Avoid diverging within a warp
  - Example without divergence:

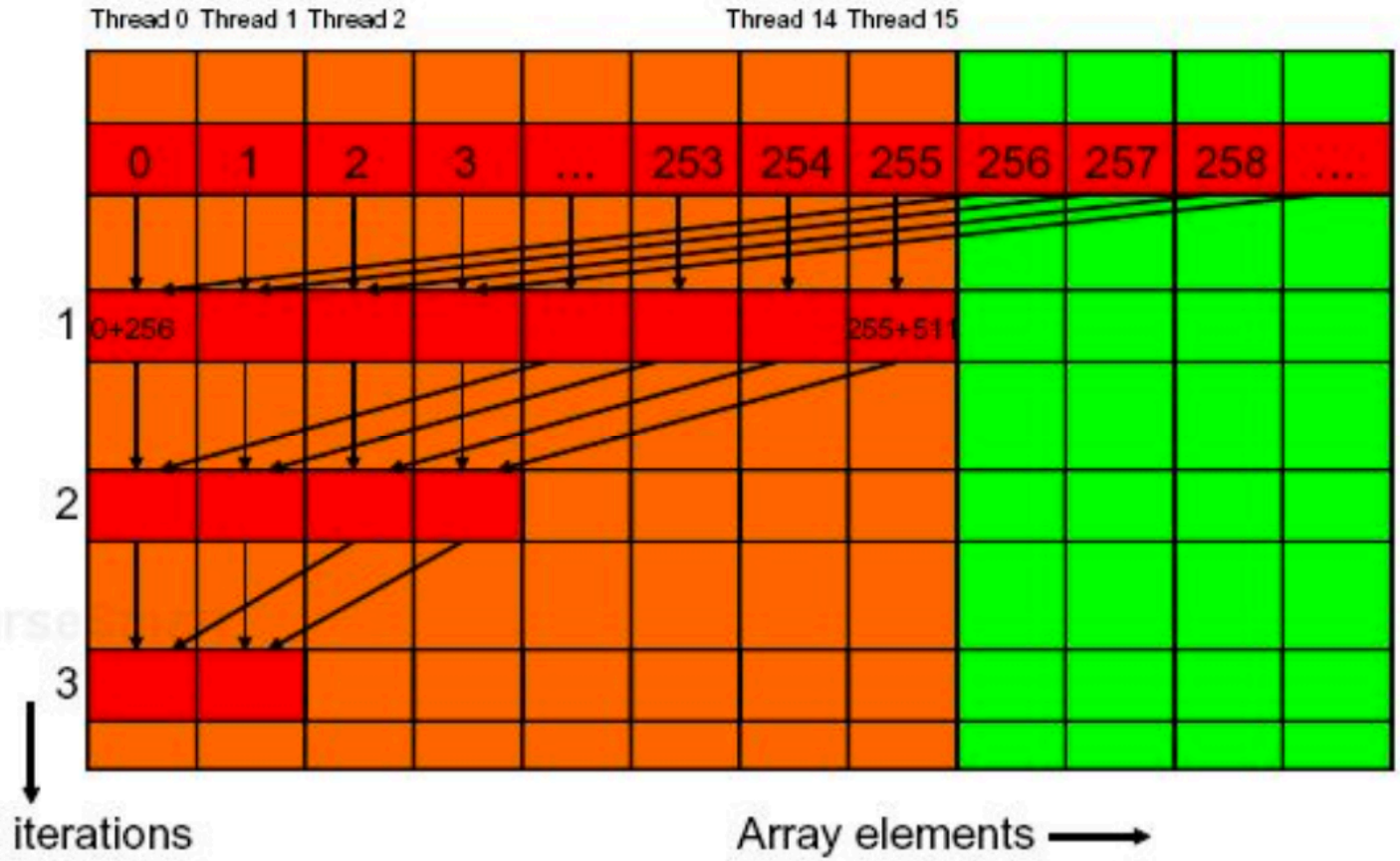


FIGURE 6.5

Execution of the revised algorithm.

Example: Divergent Iteration

```
__global__ void per_thread_sum(int *indices,
                               float *data,
                               float *sums)
{
  ...
  // number of loop iterations is data
  // dependent
  for(int j=indices[i];j<indices[i+1]; j++)
  {
    sum += data[j];
  }
  sums[i] = sum;
}
```

Iteration Divergence

- A single thread can drag a whole warp with it for a long time
- Know your data patterns
- If data is unpredictable, try to flatten peaks by letting threads work on multiple data items

Summary

- How threads are scheduled to run as warp?
- Control flow divergence

- How to avoid divergence?